



Module 2 — Sorting & Recursion

From Bubble Sort to Self-Calling Functions — a beginner-friendly journey through two foundational DSA concepts, with Java & C++ examples.

DSA SERIES

JAVA & C++

BEGINNER-FRIENDLY



Why Sort Data?

Computers find things fast when data is organized — but real-world data arrives messy. Sorting arranges elements in a specific order (ascending or descending), turning slow searches into lightning-fast ones.

🧹 The Problem

Unsorted data forces algorithms to check every single item.

🎯 The Mission

Arrange elements so Binary Search and other algorithms work efficiently.

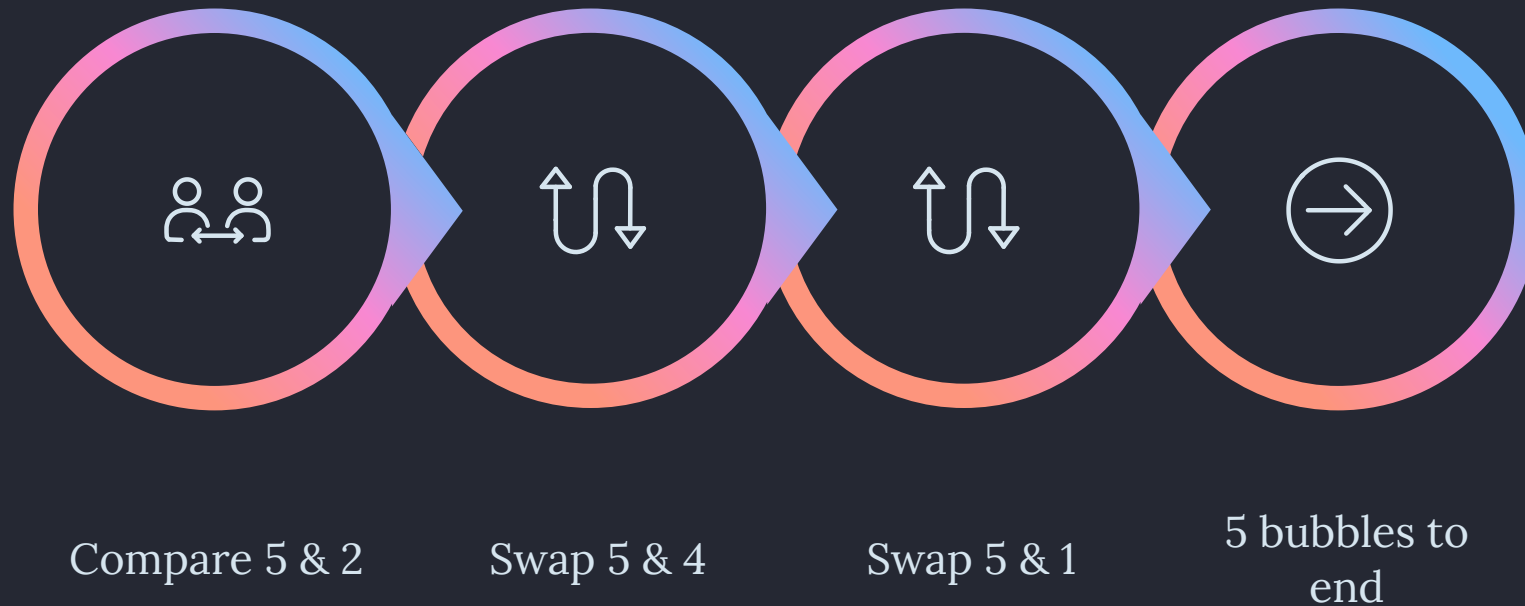
👕 The Analogy

Like organizing a chaotic laundry pile — or lining people up by height.

"If your data is sorted, your search algorithms instantly go from slow to lightning-fast."

Bubble Sort — The Mechanism

Bubble Sort compares adjacent pairs and swaps them if they're in the wrong order. The largest item "**bubbles up**" to the end on every pass.



The largest element reaches its correct position after each full pass — just like a bubble rising to the surface.

Time Complexity Analysis



Best Case — $O(n)$

The array is already sorted. One pass confirms no swaps are needed, and the algorithm stops early.

Worst Case — $O(n^2)$

The array is in reverse order. Every comparison triggers a swap, and both nested loops run fully.

❏ Bubble Sort is great for learning — but rarely used in production due to its $O(n^2)$ worst case.

Bubble Sort — Pseudocode

Using Tagalog structural names with English logic — a bilingual approach to understanding the algorithm's structure.

```
FUNCTION ayusinAngLinyahan(array)
  SET n TO length of array
  FOR i FROM 0 TO n - 1
    FOR j FROM 0 TO n - i - 2
      IF array[j] > array[j + 1] THEN
        SWAP array[j] AND array[j + 1]
      END IF
    END FOR
  END FOR
END FUNCTION
```

Outer Loop

Controls the number of passes through the array.

Inner Loop

Compares and swaps adjacent pairs on each pass.

SWAP

Exchanges two elements when they're in the wrong order.

Bubble Sort in Java & C++

JAVA

```
void ayusinAngLinyahan(int[] array) {  
    int n = array.length;  
    boolean mayPabago;  
    for (int i = 0; i < n - 1; i++) {  
        mayPabago = false;  
        for (int j = 0; j < n - i - 1; j++) {  
            if (array[j] > array[j + 1]) {  
                int temp = array[j];  
                array[j] = array[j + 1];  
                array[j + 1] = temp;  
                mayPabago = true;  
            }  
        }  
        if (!mayPabago) break;  
    }  
}
```

C++

```
void ayusinAngLinyahan(vector& array) {  
    int n = array.size();  
    bool mayPabago;  
    for (int i = 0; i < n - 1; i++) {  
        mayPabago = false;  
        for (int j = 0; j < n - i - 1; j++) {  
            if (array[j] > array[j + 1]) {  
                swap(array[j], array[j + 1]);  
                mayPabago = true;  
            }  
        }  
        if (!mayPabago) break;  
    }  
}
```

Both versions use **mayPabago** (change flag) to detect when the array is already sorted — an optimization that improves best-case performance to $O(n)$.

Recursion — Functions That Call Themselves

Recursion breaks a giant problem into smaller versions of the *exact same* problem — like Russian Nesting Dolls. To reach the tiniest doll inside, you must open each outer layer first.



Solve(4)

Depends on the result of Solve(3)

Solve(3)

Depends on the result of Solve(2)

Solve(2)

Depends on the result of Solve(1)

Base Case: Solve(1) 🎯

The stopping point — no more calls needed

Recursion — Pseudocode & Real Code

Every recursive function needs two parts: a **base case** (when to stop) and a **recursive case** (how to break the problem down).

PSEUDOCODE

```
FUNCTION magbilangPabalik(n)
  IF n == 0 THEN
    RETURN "Tapos na!"
  ELSE
    RETURN magbilangPabalik(n - 1)
  END IF
END FUNCTION
```

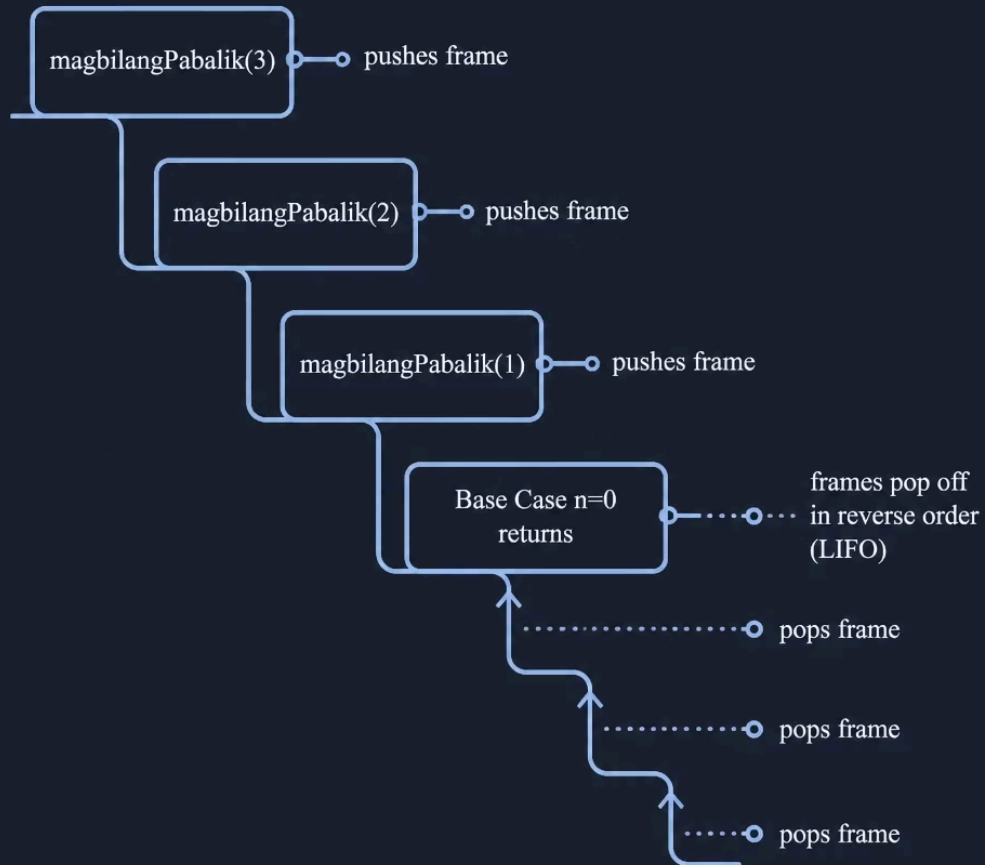
JAVA

```
String magbilangPabalik(int n) {
    if (n == 0) return "Tapos na!";
    return magbilangPabalik(n - 1);
}
```

C++

```
string magbilangPabalik(int n) {
    if (n == 0) return "Tapos na!";
    return magbilangPabalik(n - 1);
}
```


Recursion – The Stack & Common Pitfalls



How the Stack Works

Each function call creates a new **stack frame**. When the base case is reached, frames are popped off in reverse order (LIFO).

⚠ Stack Overflow

Too many nested calls without a base case will crash the program. Always ensure your recursion has a clear stopping condition!

What Comes Next

You've mastered sorting basics and recursion fundamentals. Here's the roadmap ahead:



Advanced Sorting

Quick Sort, Merge Sort — $O(n \log n)$ efficiency



Binary Trees

Hierarchical data structures for fast search



Graphs

Modeling relationships and networks



Dynamic Programming

Optimizing recursive solutions with memoization



"The best way to learn DSA is to code it yourself. Start small, break things, and iterate."